

Versión 0.2.0 | Referencia normativa

Un método normativo para la responsabilidad ingenieril, la trazabilidad y el fallo diagnosticable

Rafael Rodríguez Ramírez

Compilado 2026-01-18 12:03 UTC

Método Explícito de Ingeniería de Software



Contents

Prólogo - Contexto y motivación	4
0. El problema	5
0.1 Situación	5
0.2 Declaración del problema	5
0.3 Hipótesis	5
0.4 Objetivo	5
0.5 Alcance y no objetivos	5
1. Por qué esto es un método	6
1.1 Metodología vs método vs proceso	6
1.2 Por qué “método” encaja	6
1.3 Dependencia sin rigidez	6
1.4 Separación de responsabilidades	6
1.5 El método como función evaluativa	6
2. De software a producto de software	7
2.1 El software es salida; un producto es un resultado	7
2.2 La ingeniería hace la diferencia	7
2.3 Los productos son diagnosticables; el código por sí solo no	7
2.4 El proceso no crea productos por sí mismo	7
2.5 El papel de la explicitud	8
2.6 Resumen	8
3. Reglas normativas centrales (borrador)	9
3.0 Cómo leer estas reglas	9
3.1 Regla 1 - Declaración de intención (obligatoria)	9
3.2 Regla 2 - Explicitación de decisiones (obligatoria)	9
3.3 Regla 3 - Trazabilidad de artefactos (obligatoria)	9
3.4 Regla 4 - Requisito de validación (contextual)	10
3.5 Regla 5 - Declaración de supuestos (obligatoria)	10
3.6 Regla 6 - Compensación explícita (obligatoria)	10
3.7 Regla 7 - Conciencia de parámetros (obligatoria)	10
3.8 Regla 8 - Diagnosabilidad del fallo (regla de resultado)	10
3.9 Meta-nota	11
3.10 Conclusión	11
4. Relación con estándares	12
4.1 Compatibilidad con estándares IEEE e ISO	12
4.2 Marcos de proceso vs ejecución del método	12
5. Ortogonalidad respecto a marcos de proceso	13
5.1 Espacios de problema distintos	13
5.2 Dependencias lógicas vs prescripción del proceso	13
5.3 Coexistencia con la estructura del proceso	13
5.4 Lo que significa y no significa la ortogonalidad	13
5.5 Resumen	13
6. El fallo como resultado del método	14
6.1 Libertad de organización, no libertad de opacidad	14
6.2 El fallo no es una sorpresa	14
6.3 El fallo como evidencia, no como narrativa	14
6.4 Responsabilidad a nivel de método	14
6.5 Por qué el fallo mejora la ingeniería	15
6.6 Resumen	15
7. Esbozo de formalización - Método Explícito de Ingeniería de Software	16
7.1 Universo de discurso	16

7.1.1 Conjuntos de entidades	16
7.1.2 Predicados (tipado)	16
7.2 Relaciones núcleo (modelo de grafo)	16
7.2.1 Relaciones fundamentales	16
7.2.2 Trazabilidad (derivada)	17
7.3 Axiomas / invariantes de lógica de primer orden (FOL)	17
7.3.1 Regla 1 - Declaración de intención (obligatoria)	17
7.3.2 Regla 2 - Explicitación de decisiones (obligatoria)	17
7.3.3 Regla 3 - Trazabilidad de artefactos (obligatoria)	17
7.3.4 Regla 4 - Requisito de validación (contextual)	18
7.3.5 Regla 5 - Declaración de supuestos (obligatoria)	18
7.3.6 Regla 6 - Compensación explícita (obligatoria)	18
7.3.7 Regla 7 - Conciencia de parámetros (obligatoria)	18
7.3.8 Regla 8 - Diagnosabilidad del fallo (regla de resultado)	18
7.4 Capa deóntica (obligaciones, permisos, consecuencias)	18
7.4.1 DEBE como obligación	18
7.4.2 Compensación como permiso bajo obligación	19
7.4.3 Consecuencias (semántica de violación)	19
7.5 Cumplimiento mínimo y diagnosabilidad como funciones	19
7.5.1 Función de cumplimiento	19
7.5.2 Función de diagnosabilidad	19
7.6 Notas prácticas (no normativas)	19

Glosario**20**

Prólogo - Contexto y motivación

La mayoría de los ingenieros ha visto sistemas que funcionan y aun así se sienten frágiles. Todo compila, se despliega y pasa pruebas, y sin embargo el código parece depender de un puñado de decisiones no escritas y de expectativas frágiles. Los cambios se integran con confianza, hasta que no, y la razón es difícil de expresar sin reconstruir el pasado de memoria.

Muchas de las decisiones más consecuentes se toman bajo presión. Resuelven restricciones reales, pero con el tiempo esas elecciones se vuelven invisibles. El sistema las arrastra hacia adelante, mientras quienes las tomaron siguen su camino y el razonamiento se desvanece. Cuando ocurren fallos, las explicaciones suelen sonar como historias: quién estaba en la sala, qué plazo importaba, qué parecía razonable en ese momento.

El desarrollo de software parece maduro desde fuera. Tenemos herramientas confiables, equipos experimentados y procesos establecidos. Sin embargo, el razonamiento de ingeniería que produjo el sistema a menudo permanece implícito. El proceso es visible; la responsabilidad de ese razonamiento no. Algo importante nunca se escribió, y se siente con más fuerza cuando el sistema se tensiona.

Esto no es un problema de esfuerzo ni de inteligencia. Es un problema de visibilidad. El trabajo es real, pero el razonamiento detrás de él desaparece. Sin una forma de preservar ese razonamiento, perdemos la capacidad de entender los fallos como resultados de ingeniería y no como justificaciones narrativas.

El método descrito aquí es una respuesta a ese vacío. Busca hacer explícito el trabajo de ingeniería, preservar el razonamiento en el tiempo y hacer comprensibles los fallos en términos técnicos. No es un proceso ni un marco de trabajo. Es un método, ofrecido a quienes reconocen el problema y quieren un camino más responsable.

0. El problema

0.1 Situación

El desarrollo de software ha madurado en herramientas, coordinación y disciplina de entrega, pero persiste una brecha. A menudo se entregan sistemas que funcionan, pero no califican como productos de ingeniería responsables porque el razonamiento que los moldeó no se preserva.

Los fallos son comunes y difíciles de diagnosticar objetivamente. Las explicaciones posteriores al fallo tienden a apoyarse en narrativa más que en evidencia, incluso en organizaciones que adoptan estándares reconocidos, marcos ágiles y mejores prácticas.

0.2 Declaración del problema

La situación anterior conduce directamente a un problema central: la ausencia de representación explícita y exigible del razonamiento de ingeniería. Intención, decisiones, supuestos, trade-offs y justificación de validación permanecen implícitos o capturados de manera informal, lo que impide la evaluación, el diagnóstico y el aprendizaje objetivos.

Esto no es un problema de falta de procesos o herramientas. Los procesos coordinan el trabajo, pero por sí solos no hacen responsable el razonamiento de ingeniería.

Las consecuencias del razonamiento implícito incluyen: - decisiones que no pueden evaluarse a posteriori - fallos que no pueden diagnosticarse en términos de ingeniería - aprendizaje que permanece episódico en lugar de acumulativo

0.3 Hipótesis

Dado el problema central, la hipótesis es: si el razonamiento de ingeniería se hace explícito y exigible, independiente del proceso o del flujo de trabajo, entonces los resultados de software pueden evaluarse como productos de ingeniería y los fallos pueden diagnosticarse con responsabilidad en lugar de narrativa.

0.4 Objetivo

El objetivo es definir un método mínimo, normativo y ortogonal al proceso que especifique obligaciones y consecuencias y haga el trabajo de ingeniería evaluable objetivamente.

0.5 Alcance y no objetivos

Este trabajo no intenta reemplazar la planificación organizacional, la coordinación ni las prácticas de entrega. Distingue la responsabilidad de ingeniería de la planificación organizacional y aborda solo la primera.

La incertidumbre y la subjetividad no se eliminan; se hacen explícitas y se preservan como parte del registro de ingeniería.

Este problema no puede resolverse agregando más procesos o herramientas. Requiere un tipo diferente de intervención, lo que motiva la elección de un método como siguiente paso.

1. Por qué esto es un método

Este trabajo se enmarca deliberadamente como un método porque apunta a la responsabilidad de ingeniería, no a la coordinación de flujos de trabajo ni a la justificación académica. La distinción es operativa: determina qué puede evaluarse y cómo se puede verificar el cumplimiento en organizaciones y prácticas distintas. Un proceso puede organizar el trabajo, pero un método determina si el resultado de ingeniería es responsable.

1.1 Metodología vs método vs proceso

Un proceso es una secuencia ordenada de actividades que soporta coordinación y entrega. La metodología es el estudio y la justificación de los métodos, aportando explicaciones conceptuales y fundamento. Un marco ofrece una estructura organizacional reutilizable que empaqueta roles, prácticas y convenciones, mientras que una filosofía aporta principios guía que orientan el juicio sin imponer obligaciones. Cada uno puede mejorar la organización y el entendimiento, pero ninguno, por sí mismo, hace responsable el razonamiento de ingeniería.

Por ejemplo, un equipo puede partir de código o de modelos y aun así seguir el mismo proceso; el proceso no determina si el razonamiento detrás de esas elecciones permanece responsable.

1.2 Por qué “método” encaja

Un método aporta obligaciones normativas y consecuencias por trabajo de ingeniería faltante o insuficiente. Hace que la evaluabilidad sea independiente del orden de ejecución, de modo que el trabajo pueda evaluarse incluso cuando los equipos siguen flujos distintos o operan bajo restricciones organizacionales diferentes. Esto hace que el método sea aplicable en contextos heterogéneos sin prescribir cómo debe planificarse el trabajo.

Esta distinción importa cuando los requisitos se descubren tarde. Un proceso puede acomodar el descubrimiento tardío, pero un método determina si los cambios resultantes siguen siendo responsables.

1.3 Dependencia sin rigidez

La ingeniería tiene dependencias lógicas: un problema debe entenderse antes de validar una solución. Reconocer esas dependencias no requiere una secuencia fija de proceso. El método restringe resultados y relaciones, no el orden de actividades, y permite que los equipos descubran y revisen su entendimiento sin violar la responsabilidad.

Esto se ve cuando las decisiones se toman bajo presión. El método no las prohíbe; exige que sus consecuencias sigan siendo trazables y responsables incluso cuando el tiempo es limitado.

1.4 Separación de responsabilidades

Este enfoque no es un rechazo de Agile, Scrum, RUP ni de ningún otro enfoque organizacional. Separa responsabilidades: los procesos organizan el trabajo; un método define las obligaciones que hacen responsables los resultados de ingeniería. El método puede coexistir con cualquier enfoque organizacional porque no prescribe cómo deben coordinarse los equipos.

La separación es práctica: las decisiones organizacionales optimizan la coordinación, mientras que la responsabilidad de ingeniería requiere una base estable de evaluación sin importar el estilo de coordinación.

1.5 El método como función evaluativa

En su núcleo, un método puede actuar como una función evaluativa: dado un conjunto de artefactos de ingeniería y un contexto, puede determinar si se cumplieron las obligaciones y dónde falta responsabilidad. Esta perspectiva motiva un método por encima del proceso y prepara el terreno para distinguir software que solo funciona de software que califica como producto de ingeniería.

En conjunto, estos puntos muestran por qué un método es la abstracción correcta: preserva la responsabilidad a través de distintas formas de organizar el trabajo y hace evaluables los resultados de ingeniería. Con esa base, el siguiente paso es aclarar el límite entre software que funciona y software que califica como producto de ingeniería.

2. De software a producto de software

El software y los productos de software no son equivalentes. El software consiste en código ejecutable, configuraciones y datos que producen comportamiento. Un producto de software es el resultado de la ingeniería: software cuyo comportamiento, restricciones y propósito pueden explicarse, validarse y justificarse. Este capítulo se centra en la diferencia entre funcionalidad que funciona y resultados de ingeniería responsables.

2.1 El software es salida; un producto es un resultado

El software puede existir sin intención de ingeniería. Puede funcionar, pasar pruebas o satisfacer necesidades inmediatas y, aun así, permanecer opaco respecto a por qué existe en su forma actual, qué decisiones lo moldearon, de qué supuestos depende, qué trade-offs se aceptaron y cómo se estableció su corrección o idoneidad. Un producto de software, en cambio, es un resultado cuyas propiedades son intencionales y explicables.

Considere un pipeline de datos que ha crecido mediante arreglos urgentes. Produce salidas correctas la mayoría de los días, pero nadie puede explicar por qué existen ciertos umbrales o qué modos de fallo se aceptaron. El sistema funciona, pero su comportamiento no puede justificarse sin reconstruir decisiones pasadas de memoria.

Por contraste, un servicio de cara al cliente puede comportarse dentro de restricciones definidas y, cuando aparece una regresión de latencia, el equipo puede trazar el cambio a una decisión específica y a la condición que pretendía satisfacer. El servicio sigue fallando en condiciones reales, pero el fallo es diagnosticable como resultado de ingeniería.

La distinción no trata de tamaño, pulido, metas de rendimiento o conteo de bugs. Trata de responsabilidad de ingeniería.

2.2 La ingeniería hace la diferencia

La ingeniería introduce estructura entre problema y solución. Un producto no se define por tener más funciones o menos errores; se define por la visibilidad y la responsabilidad del razonamiento que lo moldeó. Por eso un prototipo puede ser software válido y aun así no calificar como producto: puede demostrar comportamiento sin preservar el razonamiento de ingeniería detrás de él.

Una vez que existe responsabilidad, la diagnosabilidad se vuelve una propiedad del producto en lugar de una consecuencia de quién recuerda la historia.

2.3 Los productos son diagnosticables; el código por sí solo no

Una propiedad definitoria de un producto de ingeniería es la diagnosabilidad. El éxito en producción no crea responsabilidad de ingeniería de manera retroactiva; solo muestra que el sistema es aceptable en este momento. Cuando ocurren fallos, un producto de ingeniería permite al equipo explicar el fallo en términos del razonamiento que condujo al comportamiento, en lugar de depender de narrativa retrospectiva o del recuerdo individual.

Esta distinción explica por qué la organización por sí sola no puede cerrar la brecha.

2.4 El proceso no crea productos por sí mismo

Los procesos coordinan actividades. No garantizan que el razonamiento de ingeniería se preserve. Un equipo puede seguir un proceso bien definido y aun así producir software cuyo comportamiento no puede explicarse más allá de defectos superficiales. Esto no es una crítica del proceso; es el reconocimiento de que organización y responsabilidad son preocupaciones distintas.

La diferencia es estructural, por eso requiere un mecanismo explícito.

2.5 El papel de la explicitud

La explicitud es el mecanismo mediante el cual el software se eleva a producto. Cuando el razonamiento se preserva, los resultados son intencionales, la calidad se vuelve defendible en contexto, los fallos son explicables y el aprendizaje es acumulativo en lugar de anecdótico. Esta es la diferencia estructural entre software que funciona y un producto de ingeniería.

2.6 Resumen

El software puede existir sin ingeniería; los productos no. Un producto de software se define por el razonamiento explícito, no por las mecánicas de entrega. Los procesos pueden producir software; un método define las obligaciones que permiten que el software califique como producto.

Esta distinción motiva la necesidad de obligaciones explícitas. El siguiente paso es definir cuáles son esas obligaciones, sin prescribir todavía cómo deben organizarse los equipos.

3. Reglas normativas centrales (borrador)

Estas reglas son restricciones y obligaciones normativas del método. Cada regla se enuncia para ser verificable, implicar consecuencias cuando se viola y soportar análisis sin dependencia de herramientas. Son independientes de la secuencia de trabajo y no prescriben un proceso.

3.0 Cómo leer estas reglas

Estas reglas restringen resultados, no el orden de las actividades. Un equipo puede cumplirlas mediante distintos flujos de trabajo y seguir siendo conforme, porque el método evalúa lo que existe y cómo se justifica, no cuándo se produjo.

La ausencia de artefactos solo se permite mediante compensación explícita. La compensación es un acto auditable que registra el riesgo aceptado; no elimina la obligación subyacente.

Las violaciones son condiciones diagnosticables, no fallas morales. La intención es hacer visibles y explicables las brechas para que puedan corregirse o aceptarse explícitamente.

3.1 Regla 1 - Declaración de intención (obligatoria)

Todo cambio de ingeniería no trivial DEBE declarar una Intención explícita. La Intención define el resultado, la propiedad o la restricción deseada que el cambio busca satisfacer. La Intención DEBE tener un alcance definido. Los cambios sin intención declarada se consideran trabajo implícito.

Esta regla garantiza que los cambios estén anclados a un objetivo declarado y puedan evaluarse contra él, independientemente de si el trabajo inicia desde código o desde modelos.

Ejemplo: Un equipo modifica el comportamiento de la caché con la intención de reducir la latencia para un límite de servicio definido, y registra ese alcance de forma explícita.

Justificación: Sin intención, no es posible validar ni trazar. La calidad se vuelve accidental.

3.2 Regla 2 - Explicitación de decisiones (obligatoria)

Toda decisión de ingeniería irreversible o de impacto DEBE hacerse explícita. Una Decisión DEBE referenciar: - la Intención a la que sirve - las alternativas consideradas (al menos implícitamente) Las decisiones no documentadas se tratan como Supuestos por defecto.

Esta regla preserva el razonamiento y asegura que las elecciones de impacto puedan revisarse a posteriori, incluso cuando se toman bajo presión de tiempo.

Ejemplo: Bajo un plazo de producción, un equipo elige un modelo de datos más simple y registra la decisión junto con la alternativa descartada por razones de riesgo.

Justificación: Las decisiones no documentadas impiden el análisis y el aprendizaje posteriores al fallo.



Este diagrama muestra la cadena mínima de trazabilidad que esperan las reglas: la intención informa decisiones, las decisiones moldean artefactos y la validación vincula los artefactos con su propósito previsto.

3.3 Regla 3 - Trazabilidad de artefactos (obligatoria)

Todo Artefacto que afecte el comportamiento del sistema DEBE ser trazable a al menos una Intención o Decisión. Los artefactos huérfanos se consideran artefactos injustificados. La trazabilidad DEBE ser navegable en ambas direcciones.

Esta regla garantiza que los artefactos existan por una razón localizable y revisable, en lugar de sobrevivir como residuo inexplicado de trabajo pasado.

Ejemplo: Un nuevo flag de configuración se vincula con la decisión que lo introdujo y con la intención que debía satisfacer.

Justificación: Los artefactos no trazables aumentan la complejidad sin valor responsable.

3.4 Regla 4 - Requisito de validación (contextual)

Los Artefactos y Decisiones con impacto crítico DEBEN tener Validación explícita. Lo que se considera crítico es un parámetro del método, no un umbral fijo. La Validación PUEDE adoptar distintas formas: - pruebas - revisiones - pruebas formales - simulaciones - evidencia externa

Esta regla vincula el esfuerzo de validación con la criticidad contextual, permitiendo distintas formas de evidencia mientras exige justificación explícita para trabajo de alto impacto.

Ejemplo: Un cambio de alto riesgo en un flujo de pagos se valida mediante una revisión específica y evidencia de pruebas acorde con su criticidad.

Justificación: El trabajo crítico sin validación es indistinguible de la especulación.

3.5 Regla 5 - Declaración de supuestos (obligatoria)

Los Supuestos DEBEN declararse explícitamente cuando falta evidencia o se difiere. Los Supuestos DEBEN ser identificables como tales. Los Supuestos DEBERÍAN seguirse hasta ser validados o invalidados.

Esta regla hace visible la incertidumbre y evita que premisas ocultas se conviertan en dependencias silenciosas del sistema.

Ejemplo: Un equipo asume que una dependencia externa sostendrá un throughput definido hasta que puedan ejecutarse benchmarks, y registra ese supuesto de forma explícita.

Justificación: Los supuestos implícitos acumulan riesgo en silencio.

3.6 Regla 6 - Compensación explícita (obligatoria)

Cuando falta un artefacto o validación requerida, la ausencia DEBE compensarse explícitamente. La Compensación DEBE: - estar documentada - referenciar el elemento faltante - declarar el riesgo aceptado

La compensación es un registro explícito y auditable de riesgo aceptado. No elimina la obligación; marca una desviación consciente y permite evaluar su impacto.

Ejemplo: Se despliega un hotfix sin validación completa, y el equipo registra la validación faltante, el riesgo aceptado y el plan de seguimiento.

Justificación: Se permiten atajos; no se permiten atajos silenciosos.

3.7 Regla 7 - Conciencia de parámetros (obligatoria)

El método DEBE aplicarse con parámetros contextuales explícitos. Ejemplos de parámetros: - criticidad del dominio - tolerancia al riesgo - presión regulatoria - escala del equipo - vida útil esperada del sistema

Estos parámetros influyen: - nivel requerido de explicitud - profundidad de validación - compensaciones aceptables

Esta regla asegura que las obligaciones se apliquen con contexto, de modo que la evaluación refleje el riesgo y la criticidad reales del trabajo.

Ejemplo: Un sistema en un dominio regulado declara mayor profundidad de validación y límites de compensación más estrictos que un prototipo interno.

Justificación: La calidad es contextual; el rigor sin contexto es desperdicio.

3.8 Regla 8 - Diagnosabilidad del fallo (regla de resultado)

Cuando ocurre un fallo, DEBE ser posible explicarlo en términos de la aplicación del método. Al menos uno de los siguientes DEBE ser identificable: - intención ausente - decisión inválida - artefacto no validado - supuesto falso - compensación inadecuada - parámetros mal configurados

Esta regla hace del fallo un resultado trazable de decisiones de ingeniería, no una narrativa retrospectiva. Permite analizar qué obligación no se cumplió y por qué.

Ejemplo: Después de una caída, el análisis identifica un artefacto no validado y un registro de compensación ausente como brecha decisiva.

Justificación: Si el fallo no puede explicarse en términos del método, el método no se aplicó.

3.9 Meta-nota

Estas reglas no describen un proceso. Definen restricciones y obligaciones que deben cumplirse independientemente del flujo de trabajo. Eso es lo que las convierte en reglas de un método, no de un proceso.

3.10 Conclusión

Estas reglas hacen posible la evaluación al especificar qué debe ser explícito y cómo se registran las brechas. Transforman el fallo en evidencia al hacer observables la intención, decisiones, supuestos, validación o compensación faltantes. Como restringen resultados en lugar del orden de actividad, permanecen independientes de cualquier proceso específico.

4. Relación con estándares

El Método Explícito de Ingeniería de Software está diseñado para operar dentro de estándares de ingeniería de software, no para reemplazarlos. Asume que los estándares definen el contexto de trabajo y aclara cómo se establece la responsabilidad de ingeniería dentro de ese contexto.

4.1 Compatibilidad con estándares IEEE e ISO

Los estándares de ingeniería de software IEEE / ISO definen marcos de proceso, describiendo actividades, roles y estructuras de ciclo de vida esperadas en la práctica de ingeniería. Establecen el escenario de lo que debe hacerse, pero por lo general no exigen la captura explícita del razonamiento de ingeniería ni aseguran que el fundamento se preserve en una forma analizable.

Este método complementa esos estándares al restringir resultados: exige que el razonamiento de ingeniería sea lo suficientemente explícito para soportar evaluación y diagnóstico. Usados en conjunto, los estándares definen dónde ocurre la ingeniería, mientras que el método define qué debe ser cierto para que esas actividades cuenten como ingeniería responsable.

Es posible seguir un proceso conforme a estándares y aun así producir software que no pueda diagnosticarse tras un fallo porque el razonamiento detrás de elecciones clave nunca se hizo explícito. El mismo proceso también puede producir un producto de ingeniería cuando la intención y las decisiones se capturan de forma que permitan evaluar posteriormente por qué el sistema se comporta como lo hace.

4.2 Marcos de proceso vs ejecución del método

El método es ortogonal a los estándares de proceso. Restringe resultados de ingeniería, no flujos de trabajo, y permite que el mismo proceso sea conforme o no conforme dependiendo de si se preserva el razonamiento explícito. Esta separación significa que el método no requiere cambios en fases de ciclo de vida ni en rutinas organizacionales.

Esto no es un conflicto con enfoques ágiles, planificados o híbridos. El método puede adoptarse de manera incremental en cualquiera de ellos, porque evalúa lo que existe en lugar de prescribir cómo debe ordenarse o coordinarse el trabajo.

La compatibilidad con estándares importa porque habilita diagnosabilidad y evaluación en contextos distintos. Los estándares pueden especificar actividades y artefactos, pero no pueden, por sí solos, explicar el fallo como evidencia de ingeniería. El método llena esa brecha sin alterar la estructura de trabajo definida por estándares.

El resultado es una división clara de responsabilidades: los estándares definen dónde ocurre la ingeniería, y el método define qué debe ser cierto para que cuente como ingeniería.

5. Ortogonalidad respecto a marcos de proceso

La ortogonalidad aquí es una propiedad estructural: el método evalúa el estado de ingeniería independientemente de la secuencia usada para llegar a él. La flexibilidad de orden no elimina la obligación; solo significa que hay múltiples caminos válidos hacia un resultado conforme.

5.1 Espacios de problema distintos

Los marcos de proceso organizan el trabajo mediante etapas, roles y prácticas de coordinación. Optimizan cómo los equipos planifican y entregan, pero no determinan si el razonamiento detrás del trabajo permanece responsable. El método aborda esa brecha evaluando lo que existe y cómo se justifica, independientemente de cómo se haya calendarizado el trabajo.

5.2 Dependencias lógicas vs prescripción del proceso

La ingeniería tiene dependencias lógicas, como entender un problema antes de validar una solución. La ortogonalidad no niega esas dependencias; las separa de la secuenciación fija. El trabajo real rara vez es lineal: las restricciones se descubren tarde, los supuestos se revisan y las soluciones evolucionan a medida que se profundiza la comprensión.

Un equipo puede partir de código existente, reconstruir la intención y luego registrar el razonamiento que hace defendible el comportamiento actual. Otro equipo puede partir de modelos, definir primero la intención y refinar la implementación hasta que el comportamiento coincida. Ambos pueden ser conformes si el razonamiento requerido es explícito y trazable. El método evalúa el estado al que llegaron, no la ruta que tomaron.

5.3 Coexistencia con la estructura del proceso

Un proceso puede imponer orden mientras el método permanece ortogonal. El método puede adoptarse dentro de ciclos de vida estructurados porque no exige cambios en fases ni cronogramas. El cumplimiento se evalúa sobre artefactos y trazabilidad, no sobre si un hito particular ocurrió primero.

Esto hace posible la adopción incremental: un equipo puede introducir razonamiento explícito en las partes del sistema que más importan, sin reorganizar todo el ciclo de vida.

5.4 Lo que significa y no significa la ortogonalidad

La ortogonalidad no significa que todo vale. Las obligaciones siguen aplicando, y el razonamiento faltante sigue siendo una brecha independientemente de cómo se organizó el trabajo. El método es estricto con los resultados y flexible con los caminos.

5.5 Resumen

La ortogonalidad hace que el método sea aplicable en equipos, dominios y procesos distintos porque evalúa el estado de ingeniería en lugar de la secuencia de actividades. Permite reconstrucción posterior al fallo al permitir evaluar responsabilidad incluso cuando el trabajo original fue no lineal. Esto es lo que permite que el método sea universal sin volverse prescriptivo sobre el flujo de trabajo.

6. El fallo como resultado del método

Los sistemas complejos fallan. Esto no es una anomalía; es una constante. El método no pretende eliminar el fallo. Pretende hacer que el fallo sea diagnosticable como evidencia de ingeniería y no como un artefacto narrativo.

El problema no es que ocurran fallos. El problema es que a menudo no pueden explicarse objetivamente, incluso después del hecho. Lo que distingue a la ingeniería del ensayo y error es la capacidad de explicar el fallo con evidencia trazable. El método trata el fallo como un resultado evaluable del razonamiento de ingeniería, no como una excepción inesperada.

6.1 Libertad de organización, no libertad de opacidad

Los equipos y organizaciones son libres de organizar su trabajo como consideren. El método no interfiere con modelos de coordinación, cadencia de planificación, herramientas ni estructura interna. Lo que restringe no es cómo se organiza el trabajo, sino qué debe ser explícito para que el resultado califique como ingeniería. Se preserva la libertad de organización; no la opacidad.

6.2 El fallo no es una sorpresa

La mayoría de los fallos de software son previsibles en retrospectiva. Después de un fallo, los equipos suelen reconstruir que el problema no se entendía del todo, que los supuestos resultaron falsos, que se tomaron decisiones bajo presión, que la validación fue insuficiente o que se aceptaron trade-offs en silencio. El problema no es que esas cosas ocurrieran; el problema es que no fueron explícitas.

Aquí importa la distinción entre “qué ocurrió” y “por qué esto fue posible”. Lo primero puede reconstruirse con logs y cronologías. Lo segundo requiere artefactos de razonamiento explícito que expliquen por qué se permitió que existiera un estado del sistema.

6.3 El fallo como evidencia, no como narrativa

Sin artefactos explícitos, el fallo se explica mediante narrativa: memoria, opinión y racionalización retrospectiva. Los postmortems suelen convertirse en historias sobre lo que pasó en lugar de explicaciones de por qué fue posible. Con el método aplicado, el fallo puede explicarse mediante evidencia como intención ausente, decisiones no documentadas, supuestos inválidos, validación ausente, brechas sin compensación o parámetros mal configurados. Este cambio transforma el fallo de un relato a un diagnóstico técnico basado en trazabilidad y obligaciones explícitas.

En un caso, una caída de servicio se explica como “tráfico inesperado”. La cronología es clara, pero la explicación es débil. Bajo el método, el diagnóstico apunta a una intención ausente sobre capacidad, una decisión de escalado no documentada, un supuesto inválido sobre la forma de la carga y la ausencia de validación o compensación por omitir pruebas de capacidad.

En otro caso, un defecto en etapa final se atribuye a “cambios apresurados”. La narrativa asigna la presión como causa, pero no la base de ingeniería. Bajo el método, el diagnóstico muestra que un cambio crítico salió sin validación registrada, que nunca se documentó una compensación para esa brecha y que el parámetro de tolerancia al riesgo estaba mal configurado para la vida útil esperada del sistema.

6.4 Responsabilidad a nivel de método

Cuando un sistema falla, la responsabilidad puede evaluarse al nivel del método: si se declaró y comprendió la intención, si las decisiones fueron explícitas y justificadas, si los supuestos se identificaron y se siguieron, si la validación correspondió al riesgo, si las compensaciones fueron explícitas y si los parámetros contextuales fueron realistas. Si estas preguntas no pueden responderse, el método no se aplicó plenamente. El fallo, en ese caso, no es misterioso; es esperable.

Esto no implica negligencia. Indica dónde el registro de ingeniería estuvo incompleto y dónde las obligaciones no se cumplieron o se aceptaron sin compensación explícita. El método hace visible esa brecha para poder evaluarla y corregirla.

6.5 Por qué el fallo mejora la ingeniería

Porque el fallo se vuelve explicable, se vuelve reutilizable. Los equipos pueden aprender entre proyectos, mejorar la toma de decisiones, ajustar niveles de explicitud, refinar compensaciones y evolucionar su uso del método. El fallo deja de ser un punto final y se convierte en entrada que fortalece la responsabilidad de ingeniería con el tiempo.

6.6 Resumen

El fallo es inevitable; la opacidad es opcional. El método no dicta cómo trabajan los equipos, pero sí qué debe ser explícito para que el trabajo sea ingeniería. Cuando ocurre un fallo, debe ser explicable en términos del método; la madurez de ingeniería crece a partir de fallos diagnosticados, no de fallos evitados.

El método no previene todos los fallos, y algunos fallos permanecen irreductibles o externos. Su alcance es la evaluación del razonamiento de ingeniería, no la eliminación de resultados adversos. En ese sentido, el fallo se convierte en una señal medible del estado de ingeniería, y el siguiente paso es formalizar cómo puede realizarse esa evaluación.

7. Esbozo de formalización - Método Explícito de Ingeniería de Software

Este documento proporciona un esbozo de formalización matemática y lógica del método. Es intencionalmente mínimo y se concentra en: (1) entidades, (2) relaciones, (3) obligaciones normativas y (4) diagnosabilidad. Describe restricciones sobre artefactos explícitos y relaciones, no un flujo de trabajo prescriptivo. El objetivo es evaluabilidad y diagnosabilidad, no completitud matemática.

La formalización busca **objetividad operativa bajo contexto explícito**, no objetividad absoluta.

Antes de enunciar obligaciones, debemos fijar un universo de discurso. Sin un universo declarado, las restricciones no tienen objetivos estables y la evaluación se vuelve ambigua.

7.1 Universo de discurso

Modelamos el trabajo de ingeniería como un conjunto de entidades y relaciones tipadas.

7.1.1 Conjuntos de entidades

Sea la existencia de los siguientes conjuntos:

- **I** : conjunto de *Intenciones*
- **D** : conjunto de *Decisiones*
- **A** : conjunto de *Artefactos*
- **S** : conjunto de *Supuestos*
- **T** : conjunto de *Trade-offs*
- **V** : conjunto de *Validaciones*
- **C** : conjunto de *Compensaciones*
- **Ch** : conjunto de *Cambios* (cambios de ingeniería)
- **P** : conjunto de *Parámetros* (entradas contextuales)

Definimos el conjunto global de entidades:

- **E = I D A S T V C Ch P**

7.1.2 Predicados (tipado)

Usamos predicados unarios para tipar entidades:

- Intent(x), Decision(x), Artifact(x), Assumption(x), TradeOff(x), Validation(x), Compensation(x), Change(x), Parameter(x)

El razonamiento de ingeniería se captura mediante relaciones entre estas entidades, no mediante una secuencia impuesta. Un modelo de grafo hace explícita la trazabilidad sin prescribir orden.

7.2 Relaciones núcleo (modelo de grafo)

El razonamiento de ingeniería se representa como un multigrafo dirigido tipado sobre E.

7.2.1 Relaciones fundamentales

- **declares Ch × I**
declares(ch, i) significa que el cambio ch declara la intención i.
- **serves D × I**
serves(d, i) significa que la decisión d sirve a la intención i.
- **justifies A × (I D)**
justifies(a, x) significa que el artefacto a está justificado por la intención o decisión x.

- **assumes** $(D \ A) \times S$
`assumes(x, s)` significa que la entidad x depende del supuesto s .
- **tradesOff** $(D \ A) \times T$
`tradesOff(x, t)` significa que la entidad x involucra el trade-off t .
- **validates** $V \times (A \ D \ I)$
`validates(v, x)` significa que la validación v aporta evidencia para la entidad x .
- **compensates** $C \times (I \ D \ A \ V)$
`compensates(c, x)` significa que la compensación c compensa explícitamente x faltante o insuficiente.
- **paramOf** $P \times Ch$
`paramOf(p, ch)` significa que el parámetro p aplica al cambio ch (o al alcance del proyecto que lo contiene).

7.2.2 Trazabilidad (derivada)

Definimos una relación derivada **traceableTo**(x, y) como la existencia de un camino desde x hasta y usando bordes permitidos. Como mínimo, el grafo de trazabilidad usa los bordes: - declares, serves, justifies, validates, assumes, tradesOff

Los requisitos de trazabilidad del método restringen la existencia de esos caminos y su navegabilidad.

La lógica de primer orden se usa aquí para expresar invariantes sobre el grafo. Son restricciones a evaluar, no cálculos a ejecutar.

7.3 Axiomas / invariantes de lógica de primer orden (FOL)

El método se expresa como restricciones que deben cumplirse independientemente de la secuencia de trabajo, reflejando la ortogonalidad al orden del proceso. Estas restricciones corresponden directamente a las reglas normativas definidas anteriormente.

A continuación, $\forall$ significa “para todo” y $\exists$ significa “existe”.

Las reglas 1-3 establecen la explicitud y la trazabilidad mínimas para intención, decisiones y artefactos.

7.3.1 Regla 1 - Declaración de intención (obligatoria)

Para cualquier cambio no trivial, debe declararse una intención:

- $ch \ (\text{Change}(ch) \ \text{NonTrivial}(ch) \rightarrow \exists i \ (\text{Intent}(i) \ \text{declares}(ch, i) \ \text{Scoped}(i)))$

Si un cambio no trivial no tiene intención declarada, es trabajo implícito:

- $ch \ (\text{Change}(ch) \ \text{NonTrivial}(ch) \rightarrow \exists i \ (\text{Intent}(i) \ \text{declares}(ch, i)) \rightarrow \text{ImplicitWork}(ch))$

7.3.2 Regla 2 - Explicitación de decisiones (obligatoria)

Para cualquier decisión de impacto, debe servir a una intención (o compensarse explícitamente):

- $d \ (\text{Decision}(d) \ \text{Impactful}(d) \rightarrow (\exists i \ (\text{Intent}(i) \ ? \ \text{serves}(d, i)) \ ? \ \exists c \ (\text{Compensation}(c) \ ? \ \text{compensates}(c, d))))$

Y las decisiones que no sirven a una intención se tratan como supuestos salvo que se compensen:

- $d \ (\text{Decision}(d) \ \text{Impactful}(d) \rightarrow \exists i \ (\text{Intent}(i) \ \text{serves}(d, i)) \rightarrow \exists c \ (\text{Compensation}(c) \ \text{compensates}(c, d)) \ ? \ \exists s \ (\text{Assumption}(s) \ ? \ \text{AssumptionByDefault}(d, s)))$

7.3.3 Regla 3 - Trazabilidad de artefactos (obligatoria)

Todo artefacto que afecta el comportamiento debe trazar a una intención o decisión:

- $a \ (\text{Artifact}(a) \ \text{AffectsBehavior}(a) \rightarrow (\exists x \ ((\text{Intent}(x) \ ? \ \text{Decision}(x)) \ ? \ \text{justifies}(a, x)) \ ? \ \exists c \ (\text{Compensation}(c) \ ? \ \text{compensates}(c, a))))$

Los artefactos huérfanos son injustificados:

- $a (\text{Artifact}(a) \text{ AffectsBehavior}(a) \rightarrow x ((\text{Intent}(x) \text{ Decision}(x)) \text{ justifies}(a, x)) ? \text{Orphan}(a))$

Las reglas 4-6 cubren validación, supuestos y compensación bajo riesgo contextual.

7.3.4 Regla 4 - Requisito de validación (contextual)

Los elementos críticos requieren validación, con criticidad dependiente de parámetros:

- $x (\text{Critical}(x) \rightarrow (v (\text{Validation}(v) \text{ validates}(v, x)) \text{ } c (\text{Compensation}(c) \text{ compensates}(c, x))))$

$\text{Critical}(x)$ no es fijo; se deriva de parámetros:

- $x (\text{Critical}(x) \text{ CriticalUnderParams}(x, \text{ParamsScope}(x)))$

Los parámetros son entradas de evaluación. Determinan qué cuenta como crítico y qué profundidad de validación se requiere; no excusan obligaciones.

7.3.5 Regla 5 - Declaración de supuestos (obligatoria)

Cuando falta o se difiere evidencia, los supuestos deben ser explícitos:

- $x (\text{EvidenceMissing}(x) \rightarrow s (\text{Assumption}(s) \text{ assumes}(x, s) \text{ Explicit}(s)))$

7.3.6 Regla 6 - Compensación explícita (obligatoria)

Cuando falta un elemento requerido, debe existir compensación y declarar el riesgo aceptado:

- $x (\text{Required}(x) \text{ Missing}(x) \rightarrow c (\text{Compensation}(c) \text{ compensates}(c, x) \text{ StatesAcceptedRisk}(c)))$

La regla 7 hace explícito el contexto para que la evaluación sea reproducible entre equipos y en el tiempo.

7.3.7 Regla 7 - Conciencia de parámetros (obligatoria)

Cada alcance relevante debe declarar los parámetros aplicables:

- $ch (\text{Change}(ch) \text{ NonTrivial}(ch) \rightarrow p (\text{Parameter}(p) \text{ paramOf}(p, ch)))$

Los parámetros afectan las obligaciones al definir umbrales y alcances; no son perillas de ajuste para la corrección.

La regla 8 vincula las restricciones con la diagnosabilidad, alineando el método con el análisis de fallos.

7.3.8 Regla 8 - Diagnosabilidad del fallo (regla de resultado)

Para cualquier fallo observado f , debe identificarse al menos una explicación a nivel de método:

- $f (\text{Failure}(f) \rightarrow r (\text{Reason}(r) \text{ ExplainsInMethodTerms}(r, f)))$

Razones mínimas admisibles:

$\text{ExplainsInMethodTerms}(r, f)$ implica al menos uno de: - $\text{MissingIntent}(f)$ - $\text{InvalidDecision}(f)$ - $\text{UnvalidatedArtifact}(f)$ - $\text{FalseAssumption}(f)$ - $\text{InadequateCompensation}(f)$ - $\text{MisconfiguredParameters}(f)$

7.4 Capa deóntica (obligaciones, permisos, consecuencias)

Los invariantes de FOL describen condiciones de verdad. El método es normativo: incluye obligaciones. Los operadores deónticos O, P y R son capas interpretativas sobre los invariantes; no reemplazan ni amplían las restricciones de FOL. Lo modelamos con un operador deóntico:

- **O**() : es obligatorio que se cumpla (DEBE)
- **P**() : es permitido que se cumpla (PUEDE)
- **R**() : es recomendable que se cumpla (DEBERÍA)

7.4.1 DEBE como obligación

Ejemplo (Regla 1):

- $O(\text{Change}(ch) \text{ NonTrivial}(ch) \rightarrow i (\text{Intent}(i) \text{ declares}(ch, i) \text{ Scoped}(i)))$

7.4.2 Compensación como permiso bajo obligación

El método permite atajos solo si se compensan:

- $P(\text{Missing}(x)) \text{ IF } O(c(\text{Compensation}(c) \text{ compensates}(c, x) \text{ StatesAcceptedRisk}(c)))$

Esto expresa: pueden faltar artefactos o validación, pero solo bajo compensación explícita. La compensación es permiso bajo obligación; registra riesgo aceptado y no elimina la obligación.

7.4.3 Consecuencias (semántica de violación)

Definimos un predicado de violación:

- $\text{Violation}(\text{rule_k}, \text{scope})$

Por ejemplo:

- $\neg i(\text{Intent}(i) \text{ declares}(\text{ch}, i)) \text{ implies } \text{Violation}(\text{Rule1}, \text{ch})$

Una verificación de cumplimiento es entonces una función del estado del repositorio al conjunto de violaciones:

- $\text{Violations}(\text{State}) = \{ (\text{rule_k}, \text{scope}) \mid \text{State} \text{ rule_k}(\text{scope}) \}$

Esta es la base para herramientas sin prescribir flujo de trabajo.

7.5 Cumplimiento mínimo y diagnosabilidad como funciones

7.5.1 Función de cumplimiento

Dado un estado que contiene entidades y relaciones:

- **Compliant(State)** iff $\text{Violations}(\text{State}) = \emptyset$

7.5.2 Función de diagnosabilidad

Dado un fallo f y un estado:

- **Diagnosable(f, State)** iff $\exists r \text{ ExplainsInMethodTerms}(r, f)$

Esto puede implementarse consultando el grafo explícito en busca de enlaces faltantes, validación faltante, compensación faltante o desajustes de parámetros. Es la contraparte formal del requisito de diagnosabilidad discutido antes.

7.6 Notas prácticas (no normativas)

Este nivel de formalización es suficiente para soportar evaluación objetiva, diagnóstico de fallos e implementación de herramientas sin prescribir flujo de trabajo. - Esta formalización es compatible con bases de datos de grafos y verificación de restricciones. - Es intencionalmente independiente de cualquier proceso de desarrollo o ciclo de vida específico. - Las lógicas temporales (por ejemplo, TLA+) y la semántica de ejecución pueden introducirse más adelante para modelar evolución, reconstrucción post hoc o concurrencia, pero se excluyen intencionalmente del núcleo del método.

Glosario

- Aplicación del método** La aplicación concreta del método a un alcance específico con parámetros declarados y artefactos explícitos, de modo que el cumplimiento y la diagnosabilidad puedan evaluarse.
- Artefacto** Un producto de trabajo producido, versionado e inspeccionable que afecta el comportamiento del sistema y puede validarse y trazarse a una intención o decisión.
- Compensación** Una aceptación explícita y auditable de riesgo por la ausencia de un artefacto o validación requerida, registrada sin eliminar la obligación subyacente.
- Decisión** Una elección de ingeniería comprometida entre alternativas, hecha explícita y vinculada a la intención que sirve.
- Diagnosabilidad** La capacidad de explicar un fallo en términos del método usando evidencia explícita de intenciones, decisiones, supuestos, validaciones, compensaciones y parámetros.
- Explícito** Razonamiento de ingeniería capturado como artefactos trazables con intención y validación declaradas en lugar de permanecer no documentado o implícito.
- Explicitud** La propiedad del trabajo de ingeniería de estar hecho explícito en una forma trazable, evaluable y diagnosticable.
- Fallo** Una desviación observada del comportamiento o las restricciones intencionadas o requeridas que puede analizarse contra el razonamiento de ingeniería explícito.
- Intención** El resultado, la propiedad o la restricción deseada que un cambio o sistema debe satisfacer, con un alcance definido.
- Método** Un conjunto prescriptivo y ejecutable de obligaciones y restricciones sobre artefactos y relaciones de ingeniería, parametrizable y analizable, independiente del orden del flujo de trabajo.
- Ortogonalidad** Independencia del orden del flujo de trabajo o de la secuencia del ciclo de vida, manteniendo obligaciones de ingeniería fijas.
- Parámetros contextuales** Entradas declaradas que caracterizan el contexto de ingeniería y moldean el nivel requerido de explicitud y validación para un alcance dado.
- Proceso** Una secuencia ordenada de actividades o fases usada para organizar el trabajo, que puede incorporar un método pero no define obligaciones de ingeniería por sí mismo.
- Producto de software** Software cuyo comportamiento, restricciones y propósito son responsables a través de intención, decisiones, validación y trazabilidad explícitas.
- Regla normativa** Un enunciado obligatorio que define qué debe ser explícito y las consecuencias de la omisión.
- Software** Código ejecutable, configuraciones y datos que producen el comportamiento del sistema.
- Supuesto** Un enunciado tratado como verdadero para razonar o planificar cuando la evidencia falta o se difiere, y que debe permanecer identificable hasta ser validado o invalidado.
- Trazabilidad** Vínculos explícitos y navegables entre intenciones, decisiones, supuestos, artefactos y validaciones que permiten inspeccionar y evaluar el razonamiento.
- Validación** Evidencia de que un artefacto o decisión satisface su intención declarada bajo el contexto declarado.